

Accounting App — Architecture & Code Review

Engineering Review · 16 Aug 2026

August 16, 2026

CONTENTS

Accounting App — Architecture & Code Review

1. Present state

1.1 Stack

1.2 What the frontend actually does

1.3 What the backend actually does

2. Gaps

A. Data architecture

B. Build, runtime, ops

C. Security

D. Backend completeness

E. Frontend quality

3. Upgrades & enhancements (recommended)

3.1 Must-fix before any real user (P0)

3.2 High value next (P1)

3.3 Product enhancements (P2, after the core is solid)

4. Running it

Accounting App — Architecture & Code Review

Reviewed: 16 Aug 2026. Scope: full repo (`src/` , `server/` , build/config). Verdict: **working prototype, not yet a system of record**. The UI surface is broad and genuinely useful; the persistence and accounting core are not.

1. Present state

1.1 STACK

Layer	Tech	State
Frontend	React 19, Vite 7, Tailwind 3, lucide-react, jspdf + xlsx export	Feature-rich, ~26k lines JS/JSX
Backend (live)	Express 4 + TypeScript + Prisma 5 + SQLite, JWT auth	<code>server/src/index.ts</code> , port 4001
Backend (dead)	Express + Sequelize CommonJS tree	~2.5k lines, unreachable — <code>sequelize</code> isn't even a dependency
Data store (real)	browser localStorage — key <code>accountingDB:<accountId>:<orgId></code>	Holds nearly all accounting data
Tests / CI / containers	none	—

1.2 WHAT THE FRONTEND ACTUALLY DOES

Single 10.5k-line `src/App.jsx` orchestrates everything. Modules present:

- **Sales** — invoices, estimates, credit notes, invoice preview/PDF, payment capture
- **Purchase** — bills, purchase orders, debit notes, expenses
- **Inventory** — items, stock summary, stock transfers, inter-branch transfer
- **Cash & Bank** — 1,985-line module: receipts, disbursements, bank transactions
- **Accounting** — chart of accounts, account groups/types, journal entries, ledger view
- **Reports** — Trial Balance, P&L, Balance Sheet, Cash Flow, Sales reports, **GSTR-1**, **GSTR-3B**
- **Masters/settings** — company profile, UoM, GST rates, document numbering, invoice templates, tax compliance, branches, warehouses, users, roles

- **GST engine** — `src/utils/gst.js`: 38 state codes, intra/inter-state detection, CGST/SGST vs IGST split per line

1.3 WHAT THE BACKEND ACTUALLY DOES

Only **auth + org/branch/warehouse/user/role admin + inter-branch transfers + invoices** are server-side:

- `POST /api/auth/{login,signup,setup-company,forgot-password,reset-password}`, `GET /api/auth/me`
- `/api/orgs/:orgId/{branches,warehouses,roles,users,transfers,adjustments,invoices}`
- Tenancy chain `Account -> Org -> Branch -> Warehouse`, enforced by `requireAuth -> requireTenantContext (headers x-org-id, x-branch-id, x-warehouse-id) -> requirePermission(module, action, submodule) reading DB-stored RBAC (Role / Permission / RolePermission / UserRoleAssignment)`.
- Inter-branch transfer with a real state machine (`DRAFT → SENT → RECEIVED/REJECTED`) and `StockBalance` decrement/increment inside `prisma.$transaction`.

The isolation design is the strongest part of the codebase. It's applied consistently in `branches.ts`, `warehouses.ts`, `roles.ts`, `users.ts`, `transfers.ts`, `inventoryAdjustments.ts` — **but not `invoices.ts`** (see C-2).

2. Gaps

Severity: **P0** = blocks real use / data loss / security, **P1** = serious, **P2** = quality and maintainability.

A. DATA ARCHITECTURE

#	P	Gap
A-1	P0	Books of account live in localStorage. Clearing site data, switching browser, or switching device destroys the ledger. No backup, no server copy, no export-on-write. This is the single blocking defect for an accounting product.
A-2	P0	Invoice sync is write-only. <code>createInvoiceApi/updateInvoiceApi</code> push to the server, but <code>listInvoicesApi</code> is never imported anywhere — the server copy is never read back. Two devices diverge silently; a re-install shows an empty book while the server holds rows.
A-3	P0	No general ledger. <code>buildLedgerStatement</code> (<code>src/data/db.js:3110</code>) <i>derives</i> postings at render time from invoices/bills/expenses/payments, resolving control accounts by name/code string match (<code>'accounts receivable'</code> , code <code>'1100'</code> , falling back to a suspense account). Rename an account and the trial balance changes retroactively. There is no immutable journal, no double-entry guarantee, no posted/unposted state.
A-4	P1	No fiscal-period close or lock. Any historical voucher can be edited or deleted forever; there is no “books closed through” date.
A-5	P1	No audit trail on the data that matters. <code>AuditLog</code> exists in Prisma and is written only for user CREATE/DELETE; nothing logs invoice/ledger changes.
A-6	P1	Numeric IDs come from <code>getNextNumericId</code> (max + 1 over a local array). Two tabs, or one tab per branch, mint the same ID.
A-7	P1	Money is JS <code>number</code> with <code>round2</code> at the edges. Float drift is small but real in a book that must foot exactly; server-side <code>Decimal</code> exists only for stock quantities.
A-8	P2	Voucher numbering (<code>bumpCompanyNextNumber</code>) is client-side and unenforced — the only uniqueness guard anywhere is the DB unique index (<code>orgId, branchId, number</code>) on <code>Invoice</code> .

B. BUILD, RUNTIME, OPS

#	P	Gap
B-1	P0	<code>npm start</code> on the server is broken. <code>tsconfig</code> emits ESM (<code>module: ES2022</code> , <code>moduleResolution: Bundler</code>) but leaves relative imports extensionless — <code>server/dist/index.js</code> does <code>import { buildApp } from './app'</code> , which Node rejects with <code>ERR_MODULE_NOT_FOUND</code> . Only <code>tsx</code> works. Fix: <code>module/moduleResolution: NodeNext</code> + <code>.js</code> specifiers, or bundle with <code>tsup/esbuild</code> .
B-2	P0	No containers, no CI, no deployment path (addressed here: <code>docker-compose.yml</code> , <code>docker/</code>).
B-3	P1	<code>server/.env</code> exists on disk and <code>.gitignore</code> does not cover it (it only ignores <code>*.local</code> , not <code>.env</code>). Same for <code>server/prisma/dev.db</code> . Rotate <code>JWT_SECRET</code> before this repo goes anywhere.
B-4	P1	Zero tests. No vitest/jest, no supertest, no smoke script. For tax logic (GSTR-1/3B) and ledger math, this is the highest-value missing safety net.
B-5	P1	Windows-only tooling: every <code>.vscode/tasks.json</code> task shells <code>powershell.exe / npm.cmd</code> ; helper scripts are <code>.ps1</code> . Nothing runs on this macOS host.
B-6	P2	Committed build output (<code>dist/</code> , <code>server/dist/</code>), logs (<code>server/logs/</code> , <code>*.out.log</code> , <code>*.err.log</code>), <code>tmp/</code> , and <code>_backup_before_onedrive_restore/</code> all sit in the working tree. <code>tools/bracecheck.js</code> exists purely to find unbalanced braces in <code>App.jsx</code> — a symptom, not a tool.
B-7	P2	Not a git repo. There is no undo stack for a 39k-line codebase.
B-8	P2	No lint/typecheck on the server (<code>eslint.config.js</code> covers the frontend only); frontend is untyped JS despite <code>@types/react</code> being installed.

C. SECURITY

#	P	Gap
C-1	P0	Hardcoded superuser by email. <code>src/App.jsx:9068</code> — <code>email === 'anandgowda.sr@gmail.com'</code> grants <code>isOwnerUser</code> , which disables all branch restrictions client-side. Anyone can set <code>localStorage userEmail</code> . Remove entirely; authority must come from <code>/auth/me</code> .
C-2	P0	Invoice routes have no permission check. <code>invoicesRouter.use(requireAuth, requireTenantContext)</code> only — any authenticated member of a branch can create, edit, restatus, or delete invoices regardless of role. Every other router calls <code>requirePermission</code> .
C-3	P1	Login is not account-scoped. <code>prisma.user.findFirst({ where: { OR: [{email}, {username}] } })</code> searches across all accounts, while the unique index is <code>(accountId, email)</code> — the same email can exist in two accounts, and login resolves to whichever row comes first.
C-4	P1	RBAC self-escalation path. <code>requirePermission</code> catches a denial and calls <code>ensureOwnerPermissionForCreator</code> , which <i>grants the missing permission</i> to the org creator's Owner role and retries. Convenient, but it means a permission check can mutate the permission table. Bootstrap belongs in signup/setup only.
C-5	P1	Password reset tokens live in a module-level Map (<code>RESET_TOKENS</code>) — lost on restart, not shared across instances, and generated with <code>Math.random()</code> (not cryptographic). Use <code>crypto.randomBytes</code> + a DB table.
C-6	P1	No rate limiting or lockout on <code>/login</code> , <code>/signup</code> , <code>/forgot-password</code> . No email verification on signup — every signup silently creates a new <code>Account</code> (tenant).
C-7	P1	JWT in <code>localStorage</code> (XSS-readable), 8h lifetime, no refresh, no revocation/logout server-side. <code>logout()</code> just deletes the local key.
C-8	P1	<code>POST /api/users</code> accepts <code>orgIds / branchIdsByOrg</code> from the request body and creates memberships without verifying those orgs/branches belong to the caller's <code>accountId</code> . Cross-account rows don't currently leak data (queries also filter <code>accountId</code>), but it's an unvalidated foreign key straight from user input.
C-9	P2	<code>invoices.ts</code> uses <code>\$queryRawUnsafe / \$executeRawUnsafe</code> throughout. Values are bound as parameters so it isn't injectable today, but it bypasses Prisma's typing and hardcodes SQLite syntax. Likely a workaround for a stale generated client — regenerate and use <code>prisma.invoice.*</code> .
C-10	P2	Dev-mode leaks: login distinguishes "User not found" from "Wrong password"; <code>/forgot-password</code> returns <code>devToken</code> . Both are <code>NODE_ENV</code> -gated, so ship with <code>NODE_ENV=production</code> set.

D. BACKEND COMPLETENESS

#	P	Gap
D-1	P1	SQLite only. Single-writer, no concurrency, <code>Decimal</code> emulated. Postgres needs: provider switch, <code>? → \$1</code> in the raw invoice SQL, and a real migration history (<code>prisma/migrations</code> doesn't exist — schema is applied by <code>db push</code>).
D-2	P1	No API for the bulk of the domain: customers, vendors, items, chart of accounts, journal entries, bills, payments, expenses, credit/debit notes, estimates, POs. All client-only.
D-3	P1	The <code>Item</code> and <code>StockBalance</code> tables are server-side, but the frontend's items live in <code>localStorage</code> with numeric IDs — the transfer feature therefore transfers items the server may not know about. Two disconnected item catalogs.
D-4	P2	Dead Sequelize backend (<code>server/src/{controllers,models,routes/*.js,services}</code>) duplicates <code>auth/roles/permissions</code> with a <i>different</i> permission vocabulary (<code>sales.invoices.create</code> vs <code>SALES::Invoices::CREATE</code>). Delete it, or it will be edited by mistake.
D-5	P2	<code>transfers</code> reduce stock on send with no negative-stock guard and no reservation; <code>InventoryAdjustment</code> rows are recorded but never applied to <code>StockBalance</code> .
D-6	P2	No pagination anywhere — <code>GET /invoices</code> returns every row for the branch.

E. FRONTEND QUALITY

#	P	Gap
E-1	P1	<code>App.jsx</code> is 10,554 lines holding ~35 components including full report implementations (Balance Sheet, GSTR-1, GSTR-3B). Unreviewable, unmergeable, and it re-renders the world on every keystroke — the whole DB is one <code>useState</code> serialized to <code>localStorage</code> on every change (<code>JSON.stringify</code> of the entire book, per edit).
E-2	P1	No routing (<code>react-router</code> absent). Navigation is <code>useState('dashboard')</code> , so there are no deep links, no back button, no refresh-safe state.
E-3	P1	No client-side permission gating tied to the server's permission set — <code>/auth/me</code> returns <code>isOrgAdmin</code> + <code>allowedBranchIds</code> only. Menus are hidden by ad-hoc checks.
E-4	P1	User feedback is <code>alert()</code> and <code>window.confirm</code> ; no toasts, no inline error surfaces, no optimistic/rollback handling. Failed invoice sync shows an alert and silently aborts the local save.
E-5	P2	No form validation library; no i18n; currency is hardcoded INR (<code>getCurrencyCode</code> ignores its argument). No dark mode, no responsive/mobile pass, no a11y pass.
E-6	P2	No data-grid virtualization — every list renders all rows. At the seeded 75 vouchers it's fine; at 10k it won't be.
E-7	P2	Login never sets <code>activeOrgId</code> : <code>AuthGate</code> reads <code>res.data?.companies?.[0]?.orgId</code> , but <code>POST /auth/login</code> returns only <code>{token, user}</code> . Returning users work only because the value survives in <code>localStorage</code> from the original signup on that browser. Fix on the server (return orgs) or call <code>/auth/me</code> right after login.

3. Upgrades & enhancements (recommended)

3.1 MUST-FIX BEFORE ANY REAL USER (P0)

- Move the book to the server.** Promote the `localStorage` schema to Prisma models (customers, vendors, items, accounts, vouchers, journal lines) and make the client a cache, not the source of truth.
- Introduce a real double-entry GL:** immutable `JournalEntry` + `JournalLine` rows, posted per voucher, balanced-or-rejected at write time, with control accounts referenced **by ID**, not by name. Reports read the GL.
- Close the invoice loop** — call `listInvoicesApi` on load, reconcile by `backendInvoiceId`, and make server state win.
- Delete the hardcoded owner email** (C-1) and **add `requirePermission` to invoice routes** (C-2).

5. **Fix the server build** (B-1) so a plain `node dist/index.js` boots.
6. **Ignore and rotate secrets** — add `.env`, `*.db`, `dist/`, `logs/`, `tmp/` to `.gitignore`; regenerate `JWT_SECRET`.

3.2 HIGH VALUE NEXT (P1)

- Postgres + `prisma migrate` history; `Decimal` for every monetary column.
- Split `App.jsx` by route; adopt `react-router` and lazy-load report modules.
- Server-side permission list on `/auth/me`, and a `<Can permission="...">` wrapper so UI and API agree on one vocabulary.
- Replace `alert()` with a toast/error system and add a global API error handler (401 → re-auth, 403 → explain the missing permission).
- Tests: vitest for `gst.js` /GSTR builders/ledger math, supertest for tenancy isolation (the “user of org A cannot read org B” case deserves a permanent test).
- Rate limiting (`express-rate-limit`) on auth routes, `crypto.randomBytes` reset tokens persisted in the DB, refresh-token rotation.
- Pagination + server-side filtering on every list endpoint.
- Delete the dead Sequelize tree.

3.3 PRODUCT ENHANCEMENTS (P2, AFTER THE CORE IS SOLID)

- **Compliance:** e-Invoice (IRN/QR via IRP), e-Way bill, GSTR-2B reconciliation, TDS/TCS, GSTR-1 JSON export in the government schema (today’s report is on-screen only).
- **Banking:** bank statement import (CSV/OFX) + reconciliation UI (the `bankTransactions` array already exists, unused by any importer).
- **Ops:** recurring invoices, payment reminders, customer portal / payment links, multi-currency with FX revaluation, budgets, cost centres, project accounting.
- **Platform:** background jobs (queue) for PDF/export/email, S3-compatible file storage for attachments, structured logging (pino) + request IDs, OpenTelemetry, Sentry, a `/metrics` endpoint, daily backups with a restore drill.

4. Running it

```
cp .env.docker.example .env    # then set JWT_SECRET
docker compose up --build
```

Web on `http://localhost:8080`, API on `http://localhost:4001`, SQLite persisted in the `api-data` volume. `docker compose --profile postgres up` starts Postgres too — read gap D-1 first, the schema switch is not automatic.

Local (no Docker):

```
cd server && npm install && npx prisma generate && npx prisma db push && npm run dev
```

```
npm install && npm run dev
```